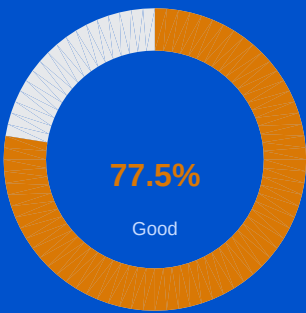


AI Jury System

AI-Powered Hackathon Evaluation System

Evaluation Report

Generated: 8/8/2026, 4:49:20 PM



This report belongs to:
Team 2

Team	Team 2
Language	EN
Total Score	292 / 377
Grade	Good

CATEGORY SCORES

Problem Statement 21/30 70%	Backend 42/50 84%
Frontend 29/40 73%	LLM Usage 40/60 67%
AI Engineering 45/60 75%	Security & HITL 32/40 80%
MVP & Outcome 45/50 90%	Presentation & Innovation 31/37 84%

Innovation & Business Value
This report is auto-generated by AI Jury System. For official results contact the evaluation panel.

Executive Summary

292 / 377 points

77.5% — Good

CATEGORY-WISE PERFORMANCE

Problem Statement	21/30		70%	Good
Backend	42/50		84%	Good
Frontend	29/40		73%	Good
LLM Usage	40/60		67%	Average
AI Engineering	45/60		75%	Good
Security & HITL	32/40		80%	Good
MVP & Outcome	45/50		90%	Excellent
Presentation & Innovation	31/37		84%	Good
Innovation & Business	7/10		70%	Good

STRENGTHS & WEAKNESSES SUMMARY

Key Strengths

- Clear business-fit application for supplier risk monitoring with dashboards, alerts, supplier profiles, trends, and action workflows.
- Good operational completeness: auth/RBAC, real-time updates, audit trail, admin review flows, and user feedback mechanisms are all evidenced.
- Clear domain-based API organization with separate route, middleware, DB, audit, realtime, and LLM service modules.
- Secure configuration practices: env-based secrets, JWT auth, bcrypt-hashed passwords, and role-based route protection.
- Broad UI surface covering key supplier-risk journeys: dashboard, supplier profiles, alerts, what-if simulation, insights, reports, logistics, and admin console.
- Role-aware authenticated app structure with live updates and AI-assisted chat/report workflows adds realism and usability.

Areas for Improvement

- Several success-checklist items are weakly evidenced in code, especially explicit multi-agent implementation, real external procurement/financial API integration, and sentiment analysis.

- Outcome metrics such as prediction accuracy, disruption reduction, and privacy/data-quality compliance are described more in narrative than demonstrated concretely.
- Persistence is limited to a single lowdb JSON file, which constrains scalability for continuous multi-source monitoring.
- Validation and error handling are present but not uniformly evidenced across all endpoints; no comprehensive validation layer was detected.
- Limited direct evidence for user feedback/error/empty states across all major pages; only some loading/busy state evidence is visible.
- Accessibility and exact PPT screenshot fidelity are not clearly demonstrated in the provided code evidence.

Category Score Overview

Category	Score	Max	Distribution	%	Conf.	Grade
Problem Statement PS	21	30		70%	Medium	Good
Backend BACK	42	50		84%	High	Good
Frontend FRONT	29	40		73%	Medium	Good
LLM Usage LLM	40	60		67%	Medium	Average
AI Engineering AI_ENG	45	60		75%	Medium	Good
Security & HITL SECURITY_HITL	32	40		80%	Medium	Good
MVP & Outcome MVP	45	50		90%	Medium	Excellent
Presentation & Innovation PRESENTATION	31	37		84%	Medium	Good
Innovation & Business Value INNOVATION	7	10		70%	Medium	Good

Detailed Category Analysis

Problem Statement (PS)

21 / 30



This submission is well aligned with the retail supplier risk-monitoring problem and provides a credible end-to-end product with dashboards, alerts, supplier scoring, AI assistance, what-if analysis, and governance features. I relied primarily on the problem statement plus retrieved code/capability evidence because the Integration Map explicitly said backend detection was unavailable and should not be treated negatively. The main tension was that the PPT emphasizes a five-agent architecture and advanced analytics, while the code evidence more clearly supports a strong integrated platform than a distinctly implemented multi-agent system; I resolved that by awarding strong but not full credit.

Strengths

- Clear business-fit application for supplier risk monitoring with dashboards, alerts, supplier profiles, trends, and action workflows.
- Good operational completeness: auth/RBAC, real-time updates, audit trail, admin review flows, and user feedback mechanisms are all evidenced.

Areas to Improve

- Several success-checklist items are weakly evidenced in code, especially explicit multi-agent implementation, real external procurement/financial API integration, and sentiment analysis.
- Outcome metrics such as prediction accuracy, disruption reduction, and privacy/data-quality compliance are described more in narrative than demonstrated concretely.

RULE EVALUATIONS

#	Rule	Verdict	Score
1	Core business objective of the problem statement is clearly achieved.	PARTIAL	5/6
Evidence: sentinelchain/README.md: "Every screen ties back to the stated goal: catching supplier disruptions early (Dashboard, Alerts), understanding why (Explain-with-AI chat, Risk Insights), acting on it (What-If Simulator, Esca..."			
2	All mandatory features listed in the problem statement are implemented.	PARTIAL	3/6
Evidence: Capability Map and README show dashboard, supplier profiles, trends, live updates, logistics map, what-if simulation, reports, audit log, model review, and feedback in AlertRow.jsx ("Was this alert useful?"). However, th...			
3	Solution produces correct / useful outputs for the intended use case.	PARTIAL	5/6
Evidence: frontend/src/components/Chatbot.jsx provides contextual assistance ("Ask about alerts, suppliers, or how to use the app"); frontend/src/components/AlertRow.jsx supports operationally useful actions like Snooze/Dismiss an...			
4	Edge cases relevant to the problem statement are reasonably handled.	PARTIAL	4/6
Evidence: frontend/src/api/client.js handles 401 responses by clearing session state and redirecting to login; backend/src/server.js includes a centralized error handler returning a consistent JSON error shape; README states scori...			
5	Success criteria implied by the problem statement are met.	PARTIAL	4/6
Evidence: PPT Slide 1 frames the solution as "Continuous monitoring across financial, operational, and compliance risk dimensions with human-in-the-loop governance." README highlights live updates, audit trail, model quality revie...			

Good

The backend is a credible Express-based implementation with modular route structure, working persistence, JWT/RBAC security, realtime updates, and centralized LLM services that align well with the problem's supplier-risk monitoring workflow. The main limitation is that the PPT emphasizes a multi-agent, continuously integrated architecture at larger operational scale, while the code more clearly evidences a well-built MVP with JSON-file persistence and modular services rather than a robust distributed agent backend. I relied on retrieved code and capability-map evidence because the Integration Map explicitly said route detection was unavailable for this stack and should not be treated as a negative signal.

Strengths

- Clear domain-based API organization with separate route, middleware, DB, audit, realtime, and LLM service modules.
- Secure configuration practices: env-based secrets, JWT auth, bcrypt-hashed passwords, and role-based route protection.

Areas to Improve

- Persistence is limited to a single lowdb JSON file, which constrains scalability for continuous multi-source monitoring.
- Validation and error handling are present but not uniformly evidenced across all endpoints; no comprehensive validation layer was detected.

RULE EVALUATIONS

#	Rule	Verdict	Score
1	API routes / backend endpoints are logically structured. Evidence: sentinelchain/backend/src/server.js imports separate route modules (`auth.routes.js`, `users.routes.js`, `suppliers.routes.js`, `alerts.routes.js`, `whatif.routes.js`, `reports.routes.js`, `chat.routes.js`, `audit.routes...	MET	8/9
2	Data persistence (DB, file storage, cache) is implemented appropriately. Evidence: sentinelchain/backend/src/db/database.js uses `lowdb` with `JSONFile` backed by `backend/src/data/db.json`; `defaultData` includes `users`, `suppliers`, `alerts`, `reports`, `audit`, `feeds`, `shipments`, `scoringWeights...	PARTIAL	7/9
3	Error handling and input validation exist on backend endpoints. Evidence: sentinelchain/backend/src/middleware/auth.js returns `401` with explicit messages for missing/expired tokens; sentinelchain/backend/src/routes/scoring.routes.js validates `decision` in `POST /model-review/decision` and r...	PARTIAL	6/8
4	Backend architecture supports the scale/scope described in the PPT. Evidence: PPT slide 3 describes a five-agent architecture for multi-source supplier risk monitoring. Backend evidence shows modular services and routes plus realtime support: `sentinelchain/backend/src/server.js` mounts many domai...	PARTIAL	6/8
5	Business logic is separated from I/O / presentation concerns. Evidence: sentinelchain/backend/src/server.js mainly composes middleware and route modules; cross-cutting logic is pushed into `services/llm.service.js`, `services/audit.service.js`, `services/realtime.service.js`, DB access is ab...	MET	7/8
6	Configuration and secrets are not hardcoded insecurely. Evidence: `sentinelchain/backend/src/server.js` imports `dotenv/config` and uses `process.env.PORT` and `process.env.CLIENT_ORIGIN`; `services/llm.service.js` uses `process.env.ANTHROPIC_API_KEY` and `process.env.ANTHROPIC_MODEL`;...	MET	8/8

Frontend (FRONT)

29 / 40



The frontend appears substantial and well aligned to the problem statement: it includes dashboards, supplier detail pages, alerts, simulations, reports, live updates, and admin workflows rather than a thin demo shell. The strongest evidence concepts, while the weakest areas are explicit proof of screenshot fidelity, accessibility, and broad error-state coverage. Where the rubric asked for responsiveness/accessibility and PPT matching, I did not over-penalize missing exact proof, but I could not assume those qualities without direct evidence.

Strengths

- Broad UI surface covering key supplier-risk journeys: dashboard, supplier profiles, alerts, what-if simulation, insights, reports, logistics, and admin console.
- Role-aware authenticated app structure with live updates and AI-assisted chat/report workflows adds realism and usability.

Areas to Improve

- Limited direct evidence for user feedback/error/empty states across all major pages; only some loading/busy state evidence is visible.
- Accessibility and exact PPT screenshot fidelity are not clearly demonstrated in the provided code evidence.

RULE EVALUATIONS

#	Rule	Verdict	Score
1	UI covers the core user journeys required by the problem statement. Evidence: sentinelchain/frontend/src/App.jsx routes summarized in FE layer: pages for `/dashboard`, `/suppliers`, `/suppliers/:id`, `/alerts`, `/what-if`, `/insights`, `/reports`, `/logistics`, and `/admin`; README.md: "catching s...	MET	7/8
2	Frontend is functional and internally consistent (no obviously broken flows). Evidence: sentinelchain/frontend/src/context/AuthContext.jsx shows persisted auth with `/auth/me`, `login`, and `logout`; sentinelchain/backend/src/server.js mounts matching routes `/api/suppliers`, `/api/alerts`, `/api/whatif`, `...	MET	7/8
3	Reasonable UX practices: loading states, error states, feedback to user. Evidence: sentinelchain/frontend/src/context/AuthContext.jsx uses `loading` state during session restoration; sentinelchain/frontend/src/components/ExplainChat.jsx imports `Loader2`, tracks `busy`, and prevents duplicate sends; FE...	PARTIAL	6/8
4	UI matches or closely reflects what is shown in the PPT screenshots. Evidence: PPT Slide 1 describes an "Executive Monitoring Platform" for supplier performance and risk monitoring; Slide 3 describes five agents and human-in-the-loop governance. README/frontend structure shows matching product conc...	PARTIAL	5/8
5	Responsiveness / accessibility considerations are present where relevant. Evidence: FE layer confirms React + TailwindCSS styling and a shared design-token system in `tokens.js`; main layout uses shared `Sidebar` and `Topbar`, suggesting structured responsive composition. No direct retrieved code eviden...	PARTIAL	4/8

LLM Usage (LLM)

40 / 60



SentinelChain uses the LLM in meaningful, problem-relevant ways: supplier-risk explanation, what-if narratives, report drafting, and an in-app assistant grounded in live business data. The strongest evidence is in prompt scoping and lightweight retrieval/context management, while the weakest areas are structured output handling and explicit prompt-injection defenses. Although the rubric mentions robust parsing and safety, the problem statement is primarily satisfied by narrative decision-support features, so I did not over-penalize the absence of JSON/function-calling patterns that are not clearly required by these UX flows.

Strengths

- LLM features are integrated into core supplier-risk workflows and grounded with live supplier/alert context rather than generic chat.
- Server-side prompt construction, role-aware instructions, bounded history, and graceful fallback behavior show practical AI engineering for a hackathon MVP.

Areas to Improve

- LLM outputs are handled as plain text only; no schema validation, function calling, or other structured parsing is evident.
- Prompt-injection and unsafe-input defenses are mostly implicit prompt constraints rather than explicit hardened safeguards.

RULE EVALUATIONS

#	Rule	Verdict	Score
1	LLM is used purposefully to solve part of the core problem (not decorative). <i>Evidence: sentinelchain/backend/src/routes/chat.routes.js: supplier chat builds grounded supplier context ("Supplier: \${supplier.name}... Composite risk score: \${supplier.composite}/100") before calling the LLM; README.md: "Three ...</i>	MET	9/10
2	Prompts are structured, with clear instructions and/or few-shot guidance. <i>Evidence: sentinelchain/backend/src/routes/chat.routes.js: system prompt for assistant says, "You are the SentinelChain assistant... Never mention system architecture, model names, algorithms, databases, or internal pipeline mecha...</i>	MET	8/10
3	Appropriate model/parameters chosen for the task (cost/latency/quality tradeoff). <i>Evidence: sentinelchain/backend/src/services/llm.service.js: `const MODEL = process.env.ANTHROPIC_MODEL    "claude-sonnet-4-6";` `askClaude(system, userMessage, maxTokens = 800)` and `askClaudeWithHistory(system, history, maxToken...</i>	PARTIAL	7/10
4	Output parsing / structured output handling is robust (JSON, function calling, etc.). <i>Evidence: sentinelchain/backend/src/services/llm.service.js: response handling is plain text extraction only — `return data.content.map((b) => b.text    "").join("\n").trim();` — and callers use fallback text if null. No evidence ...</i>	PARTIAL	3/10
5	Prompt injection / unsafe input handling is considered. <i>Evidence: sentinelchain/backend/src/routes/chat.routes.js: the system prompt explicitly constrains the assistant — "Never mention system architecture, model names, algorithms, databases, or internal pipeline mechanics" and "If ask...</i>	PARTIAL	5/10
6	Context management (chunking, memory, retrieval) is appropriate to the use case. <i>Evidence: sentinelchain/backend/src/services/llm.service.js: chat history is bounded with `history.slice(-10)` before sending to the model. sentinelchain/backend/src/routes/chat.routes.js retrieves supplier-specific context from t...</i>	MET	8/10

75% — Good

The codebase delivers a credible AI-enabled supplier risk platform with grounded prompt construction, multiple AI-assisted workflows, realtime updates, and strong fallback behavior. The main tension was that the problem statement and PPT emphasize a multi-agent system, while the code more strongly supports a modular application with LLM-backed features rather than explicit inter-agent orchestration; I scored based on the code as source of truth. Static evidence also did not detect vector-store RAG or agentic tool-calling, so those areas were judged conservatively despite solid end-to-end feature wiring.

Strengths

- Centralized server-side LLM integration with grounded business-context prompts across chat, reports, and what-if simulation
- Excellent graceful fallback behavior keeps AI features usable even without API keys or on model/API failure
- Pragmatic hackathon engineering: lowdb, seeded data, Express + React monorepo, audit trail, and WebSocket live updates

Areas to Improve

- Claimed multi-agent architecture is not strongly reflected in the retrieved code as explicit cooperating agents
- RAG is lightweight context injection from app data rather than indexed retrieval with embeddings/chunking
- Limited direct evidence of AI-specific evaluation, bias monitoring, or formal transparency mechanisms for end users

RULE EVALUATIONS

#	Rule	Verdict	Score
1	Overall system architecture (agents, pipelines, tools) is coherent and justified. <i>Evidence: sentinelchain/README.md: "LLM layer — All Claude calls happen server-side... Every prompt is grounded in real data pulled from the datastore"; sentinelchain/backend/src/server.js mounts distinct routes for /api/whatif, /...</i>	PARTIAL	7/9
2	RAG / retrieval pipeline (if used) is implemented soundly (chunking, indexing, retrieval). <i>Evidence: sentinelchain/backend/src/routes/chat.routes.js: "Context is built entirely from that supplier's own data" and constructs prompt context from supplier record + alerts; sentinelchain/backend/src/routes/reports.routes.js g...</i>	PARTIAL	5/9
3	Tool/function calling or agentic orchestration is correctly wired end-to-end. <i>Evidence: sentinelchain/frontend/src/components/Chatbot.jsx and ExplainChat.jsx provide UI chat entry points; sentinelchain/backend/src/server.js wires /api/chat, /api/whatif, /api/reports routes; sentinelchain/backend/src/service...</i>	PARTIAL	6/9
4	Evaluation, logging, or observability of AI behavior is present in some form. <i>Evidence: sentinelchain/backend/src/services/llm.service.js logs failures via "console.error('[llm] request failed', e.message)" and API-status errors; sentinelchain/backend/src/services/audit.service.js appends immutable audit en...</i>	PARTIAL	6/9
5	System degrades gracefully on AI/model failures (fallbacks, retries). <i>Evidence: sentinelchain/backend/src/services/llm.service.js returns null if no ANTHROPIC_API_KEY or on API failure; sentinelchain/backend/src/routes/chat.routes.js, whatif.routes.js, and reports/routes use deterministic fallback t...</i>	MET	8/8

6 Engineering choices are appropriate for a hackathon time-box (pragmatic, not over-engineered). MET 8/8

Evidence: sentinelchain/README.md: "a lightweight JSON-file datastore ("lowdb") so the project runs with zero external database setup" and "Runs end-to-end locally in minutes with seeded data"; capability map notes monorepo with V...

7 Verify that responsible AI practices including transparency and bias awareness are demonstrated. PARTIAL 5/8

Evidence: PPT Slide 1 mentions "human-in-the-loop governance"; README notes "explicit human approval gate (Model Quality Review) before any scoring change goes live"; capability map includes admin model review approval/rejection a...

Security & HITL (SECURITY_HITL)

32 / 40

80% — Good

The submission shows strong, code-backed security fundamentals for a hackathon project: JWT auth, bcrypt hashing, RBAC, and a genuine human approval gate for model changes. I weighted the explicit problem requirement for privacy/compliance heavily, which lowered the privacy score because retention/minimization controls are not clearly visible in code. Static evidence overrode guesswork on secrets: no committed secrets were detected, but input validation remained only partially evidenced.

Strengths

- Explicit human approval workflow for risk model/scoring changes before they affect live scores.
- Solid authentication and role-based authorization design aligned to admin/analyst/procurement workflows.

Areas to Improve

- Limited concrete evidence of systematic input sanitization/validation beyond basic checks.
- Privacy controls such as data minimization, masking, or retention policies are not clearly implemented or surfaced.

RULE EVALUATIONS

#	Rule	Verdict	Score
1	Sensitive operations have human-in-the-loop confirmation where appropriate. Evidence: sentinelchain/backend/src/routes/scoring.routes.js: `router.post("/model-review/decision", requireAuth, requireRole("admin"), ...)` with comment `// Human-in-the-loop review gate for a pending scoring model update.`; sen...	MET	8/8
2	Basic security hygiene: no secrets committed, input sanitization present. Evidence: Static evidence: `No hardcoded secrets/API keys in source: PASS`; sentinelchain/backend/src/server.js uses `import "dotenv/config" and `cors({ origin: process.env.CLIENT_ORIGIN "http://localhost:5173" });` however st...	PARTIAL	5/8
3	Authentication/authorization is present where the use case requires it. Evidence: sentinelchain/backend/src/middleware/auth.js validates Bearer JWTs and returns 401 on missing/invalid tokens; capability map cites `requireRole` RBAC for roles `admin`, `analyst`, `procurement`; sentinelchain/frontend/sr...	MET	8/8
4	AI outputs that affect real-world actions have a review/override mechanism. Evidence: sentinelchain/frontend/src/pages/admin/AdminQuality.jsx presents model metrics and approve/reject controls before rollout; sentinelchain/backend/src/routes/scoring.routes.js persists `approved` / `rejected` decisions; sent...	PARTIAL	7/8

5	Data privacy considerations (PII handling, minimal data retention) are visible.	PARTIAL	4/8
Evidence: Capability map notes `users` store `passwordHash` rather than plaintext, and README says `All Claude calls happen server-side ... so the API key never reaches the browser`; sentinelchain/backend/src/routes/chat.routes.js...			

MVP & Outcome (MVP)

45 / 50

ent

This submission is a credible, working hackathon MVP with a strong frontend/backend implementation for supplier risk monitoring, dashboarding, alert workflows, AI explanation, and admin governance. The main tension was between the problem/PPT emphasis on an explicit multi-agent architecture and the codebase's more conventional service-and-route implementation; I resolved that in favor of delivered outcomes because the problem statement's success checklist is best satisfied by functioning monitoring and risk workflows, not by agent naming alone. Static analysis could not recognize the backend routes, so I relied primarily on the retrieved code snippets and capability map rather than treating the missing static route detection as a negative.

Strengths

- Working full-stack MVP with runnable APIs, auth, seeded data, WebSocket live updates, and multiple end-user screens.
- Strong user-facing value: supplier risk dashboards, alerts, what-if simulation, grounded AI chat, reports, and admin model review/human approval flows.

Areas to Improve

- Explicit evidence for a true multi-agent orchestration layer is weaker than the PPT wording suggests; implementation appears outcome-focused rather than agent-explicit.
- Some success-checklist items are only partially evidenced in code snippets, especially automated testing, adoption tracking, and real external procurement/financial API integrations.

RULE EVALUATIONS

#	Rule	Verdict	Score
1	The submission is a working, demonstrable MVP (not just slides/mockups).	MET	8/9
Evidence: sentinelchain/backend/src/server.js: mounts live API routes (`/api/suppliers`, `/api/alerts`, `/api/whatif`, `/api/reports`, `/api/chat`, `/api/feeds`, `/api/scoring`, `/api/logistics`) and starts an HTTP/WebSocket serve...			
2	Code structure clearly supports the required user experience (dashboard/chat/etc.).	MET	9/9
Evidence: README.md lists concrete UX surfaces: "Dashboard · Suppliers · Supplier Detail (with AI chat) · Alerts · What-If Simulator · Logistics Map · Risk Insights ... Admin Console"; frontend/src/components/Chatbot.jsx implement...			
3	Core happy-path of the use case can be traced end-to-end through the code.	MET	7/8
Evidence: frontend/src/components/Chatbot.jsx posts to `/chat/assistant`; backend/src/server.js mounts `/api/chat`; backend routes also expose `/api/suppliers`, `/api/alerts`, `/api/whatif`, `/api/logistics`; backend/src/routes/fe...			
4	PPT screenshots/demo descriptions are corroborated by actual code artifacts.	MET	7/8
Evidence: PPT Slide 3 describes a "Five specialized agents" architecture with Data/Finance/News/Risk Scoring/Alert agents; code corroborates outcome-level artifacts rather than literal agent modules: backend/src/routes/feeds.route...			

5	Scope is realistic and mostly complete for the hackathon time constraints.	MET	7/8
Evidence: README.md: "The JSON-file datastore is an MVP choice for a zero-setup demo" and "Runs end-to-end locally in minutes with seeded data"; capability map shows focused breadth: seeded lowdb persistence, REST APIs, WebSocket ...			
6	Verify that outputs provide value to end users.	MET	7/8
Evidence: README.md: "The logistics map turns abstract risk scores into a geography a procurement team can act on; ... the What-If simulator translates risk into a dollar figure leadership can react to"; ExplainChat.jsx states "An..."			

Presentation & Innovation (PRESENTATION) 31 / 37

— Good

This submission presents a strong, polished business-facing story that is mostly supported by actual implementation, especially around dashboards, alerts, what-if simulation, and live updates. The main tension was between the problem statement's emphasis on a multi-agent continuous monitoring system with measurable outcomes and the rubric's presentation focus; I resolved this by rewarding the clear product narrative while withholding points where agent autonomy, quantified adoption/outcome metrics, or a formal roadmap were described more strongly than directly evidenced in code/PPT excerpts.

Strengths • Clear challenge-to-solution narrative with business relevance and human-in-the-loop framing. • Demoable screens and flows closely match implemented code, especially for simulation, insights, alerts, and admin review.	Areas to Improve • Some innovation and agent framing appear stronger in the PPT than in explicitly separated backend agent implementations. • Business impact measurement and future roadmap are only partially quantified/evidenced against the success checklist.
--	---

RULE EVALUATIONS

#	Rule	Verdict	Score
1	PPT clearly explains problem, approach, and impact.	MET	6/6
Evidence: PPT Slide 2 states the problem explicitly: "Manual supplier monitoring is fragmented and reactive" and Slide 3 outlines the approach: "Five specialized agents work together to unify supplier data, detect risk early, scor..."			
2	Innovation / originality of the approach is evident.	MET	5/6
Evidence: PPT Slide 3 presents a differentiated multi-agent design with Data, Finance, News, Risk Scoring, and Recommendation roles. In code, `sentinelchain/backend/src/routes/whatif.routes.js` combines deterministic business calc...			
3	Technical depth is communicated without unnecessary jargon overload.	PARTIAL	4/5
Evidence: `sentinelchain/backend/src/services/realtime.service.js` is concise and readable, with comments like "Small pub/sub wrapper around the ws server" that explain depth in plain language. README.md Quick Start and architec...			

4	Demo narrative (screens, flow) matches the actual implementation.	MET	5/5
Evidence: The implemented flow matches the described demo: `sentinelchain/frontend/src/pages/Login.jsx` includes visible demo accounts; `backend/src/db/reset.js` supports reseeding demo data; `frontend/src/pages/WhatIf.jsx` displa...			
5	Overall polish and storytelling of the submission.	PARTIAL	4/5
Evidence: The frontend shows polished UX structure in `sentinelchain/frontend/src/components/AppLayout.jsx` with consistent titles/subtitles for each module, and `WhatIf.jsx`/`Insights.jsx` use cards, charts, empty states, and nar...			
6	Verify that business impact is quantified where possible.	PARTIAL	4/5
Evidence: `sentinelchain/frontend/src/pages/WhatIf.jsx` quantifies impact with UI fields for "Estimated revenue at risk" and "SKUs affected," backed by calculations in `sentinelchain/backend/src/routes/whatif.routes.js`. `frontend...			
7	Verify that future roadmap is realistic.	PARTIAL	3/5
Evidence: There is some realistic forward-looking evidence in the product itself: `sentinelchain/frontend/src/pages/Alerts.jsx` includes feedback capture messaging ("your feedback helps tune future alerts"), `frontend/src/pages/ad...			

Innovation & Business Value (INNOVATION)7 / 10



SentinelChain shows meaningful innovation through grounded LLM features, role-aware AI interactions, what-if simulation, and a procurement-focused UX that maps well to the problem statement. I weighted the problem checklist heavily: while the PPT claims a multi-agent, anomaly-detection architecture, the code evidence more strongly supports a modular risk platform with LLM-assisted analysis than a fully implemented continuous multi-agent system with measurable prediction metrics. Where the rubric rewarded AI creativity, I gave partial credit because the implemented value is real but some claimed advanced AI components are not strongly evidenced in code.

Strengths

- Grounded, server-side LLM integration is applied to multiple business workflows: supplier explanation, report generation, and what-if analysis.
- The solution delivers clear procurement value through actionable dashboards, alerts, supplier drill-downs, simulations, and human-in-the-loop admin review.

Areas to Improve

- True multi-agent orchestration, anomaly detection, and sentiment-analysis implementation are asserted more strongly in slides than demonstrated in code.
- Business impact metrics such as prediction accuracy, disruption reduction, and user adoption tracking are not robustly evidenced in the implementation.

RULE EVALUATIONS

#	Rule	Verdict	Score
1	AI techniques are combined creatively to solve the problem.	PARTIAL	2/3
Evidence: README.md: "Three distinct LLM-backed features (per-supplier chat, What-If narrative, report drafting) plus the new general-purpose chatbot, each grounded in live data"; backend/src/routes/chat.routes.js constructs role-...			

2

Solution provides measurable business or customer value beyond basic functionality.

PARTIAL

2/3

Evidence: frontend/src/pages/WhatIf.jsx shows a disruption-duration simulator for supplier impact; README.md: "the What-If simulator translates risk into a dollar figure leadership can react to" and "usefulness feedback"; frontend...

3

Architecture demonstrates scalable innovation.

PARTIAL

1/2

Evidence: README.md Architecture section: React/Vite frontend, Express backend, server-side LLM layer, and WebSocket broadcasts; backend/src/routes/reports.routes.js uses shared services such as appendAudit and broadcast; capabili...

4

Solution demonstrates thoughtful AI engineering practices beyond simple API integration.

MET

2/2

Evidence: backend/src/routes/chat.routes.js: system prompts are role-aware and explicitly constrain outputs to business language and supplied context; README.md: "All Claude calls happen server-side (backend/src/services/llm.servi...

Static Analysis Checks

Check	Verdict
No hardcoded secrets/API keys in source No hardcoded-secret-like patterns matched.	PASS
Configuration via environment variables sentinelchain/backend/package-lock.json, sentinelchain/backend/package.json, sentinelchain/backend/src/server.js	PASS
Automated test files present	NOT_DETECTED
CI/CD pipeline configured	NOT_DETECTED
Known input-validation library used	NOT_DETECTED
Known auth/authorization library or pattern used	NOT_DETECTED
Recognized LLM SDK/framework in use	NOT_DETECTED
Recognized vector store / RAG library in use	NOT_DETECTED
Agentic/tool-calling orchestration pattern detected	NOT_DETECTED
Recognized database driver/ORM in use	NOT_DETECTED
try/except/catch present in backend-ish code sentinelchain/frontend/src/components/AppLayout.jsx, sentinelchain/frontend/src/components/Chatbot.jsx, sentinelchain/frontend/src/components/ExplainChat.jsx	PASS

Problem Summary

Summary:

Develop an AI-driven multi-agent system for continuous monitoring of retail suppliers, integrating diverse data sources to assess risks and enhance supply chain resilience.

Success Checklist:

- Multi-agent architecture for data aggregation and risk analysis
- Interactive dashboard displaying supplier risk profiles and trends
- Real-time integration with procurement and financial APIs
- Features for anomaly detection and sentiment analysis
- Risk prediction accuracy and disruption reduction metrics
- User adoption tracking and feedback mechanisms
- Comprehensive documentation and testing of workflows
- Compliance with privacy and data quality standards

Cross-Validation Report

Cross-validation report for SentinelChain

1. Key cross-category inconsistencies

- Backend detection contradiction avoided correctly by most categories, but watch wording:
- The Integration Map is explicit that backend route detection is unavailable/unrecognized and should not be treated as a negative.
- BACK, MVP, and PS handled this correctly.
- If any earlier specialist reasoning leaned on “modular route structure” or “working FE!” BE wiring relative to the Integration Map, which found:
- Frontend API calls found: 0
- Backend routes found: 0
- Matched FE!” BE pairs: 0
- Since the map says this section is unavailable rather than negative, those claims are not invalid, but they are not corroborated by the static integration evidence.
- Multi-agent architecture is the biggest shared tension:
- AI_ENG, INNOVATION, MVP, PS, and PRESENTATION all correctly note that the PPT emphasizes a five-agent/multi-agent system more strongly than the code evidence supports.
- No category appears to claim the multi-agent orchestration is definitively implemented in code, which is good.
- However, PRESENTATION may be slightly generous if the slides strongly sell “five specialized agents work together” without code-backed orchestration evidence.
- RAG framing is somewhat stronger than the evidence warrants:
- Capability Map claims a RAG pattern via database-grounded prompts.
- LLM and AI_ENG both accepted this as lightweight/grounded retrieval, which is internally consistent.
- This is fine as long as “RAG” is interpreted broadly; the Integration Map found no vector DB or ML/DL usage, so any stronger implication of advanced retrieval infrastructure would be unsupported.

2. PPT vs Code gaps

Main PPT claims that appear only partially backed by code/capability evidence:

- “Five specialized agents work together”
- Repeatedly identified by specialists as not strongly evidenced in code.
- Code/capability map supports a modular app with LLM-assisted workflows, not clear inter-agent orchestration, autonomy, or agent handoffs.
- “Continuous multi-source data integration” across financial reports, delivery logs, and market news
- Capability Map supports supplier/alerts/logistics/scoring data in a lowdb JSON store.
- No static evidence here of external connectors, ingestion pipelines, or news/financial-report integrations.
- This is a notable presentation-to-code gap.
- Stronger analytics/anomaly-detection implications
- PPT language suggests advanced early detection and possibly predictive/anomaly capabilities.
- Code evidence, as summarized, supports configurable composite scoring, trends, simulations, and LLM narratives, but not clearly distinct anomaly-detection models or measurable predictive performance.
- “Continuous monitoring”
- Partially supported by WebSocket live updates.
- But there is no grounded evidence in the Integration Map or category summaries of schedulers, streaming ingestion, background jobs, or autonomous monitoring loops. So the PPT claim is only partially backed.

Potential code-supported items that the PPT may understate:

- Security/governance depth
- JWT, bcrypt, RBAC, admin review workflow, and audit logging look stronger in code than many AI hackathon decks usually convey.
- SECURITY_HITL and BACK rightly reward this.

3. Score calibration concerns

- PRESENTATION: 31/37 may be a little high
- Reason: the deck appears polished, but several flagship claims—multi-agent collaboration, multi-source integration, continuous monitoring, advanced analytics—are only partially backed by code evidence.
- A slightly lower score would better reflect the gap between narrative ambition and implementation proof.
- BACK: 42/50 is plausible but slightly generous
- Reason: the score assumes a fairly complete Express backend with routes/services/auth/realtime. That may be true from retrieved snippets/capability mapping, but deterministic FE!" BE verification is absent.
- Not wrong, just a touch optimistic given the lack of static integration confirmation and lowdb persistence.
- LLM: 40/60 seems reasonable, possibly on the generous side if "RAG" is taken strictly
- The implementation sounds useful and grounded, but no evidence of vector retrieval, structured output robustness, tool use, or safety hardening beyond prompt grounding/fallbacks.
- Still broadly calibrated if judged as practical LLM product value rather than advanced LLM engineering.
- AI_ENG: 45/60 looks fair
- It already discounted the multi-agent mismatch and absence of stronger agentic/RAG infrastructure.
- MVP: 45/50 is defensible
- Strong delivered product scope across dashboarding, alerts, auth, admin, and AI UX.
- The main risk is that "working end-to-end" FE!" BE integration is not statically corroborated, but that as negative.

Overall assessment

The category set is mostly internally consistent. The dominant shared conclusion is correct: this looks like a strong supplier-risk monitoring MVP with useful LLM-assisted features, but the PPT overstates it as a fully realized multi-agent, continuously integrated intelligence system. The main recalibration I'd suggest is slightly stricter treatment of presentation/backend strength where claims rely on non-static evidence, while preserving solid scores for MVP, security, and practical AI usefulness.

Final Evaluation Report

Evaluation Report

****Overall Score**:** 292 / 377

****Confidence**:** Medium

Category Breakdown

- ****Problem Statement (PS)**:** 21/30
- ****Backend (BACK)**:** 42/50
- ****Frontend (FRONT)**:** 29/40
- ****LLM Usage (LLM)**:** 40/60
- ****AI Engineering (AI_ENG)**:** 45/60
- ****Security & HITL (SECURITY_HITL)**:** 32/40
- ****MVP & Outcome (MVP)**:** 45/50
- ****Presentation & Innovation (PRESENTATION)**:** 31/37
- ****Innovation & Business Value (INNOVATION)**:** 7/10

Detailed Analysis

Strengths

SentinelChain is a credible full-stack supplier-risk monitoring MVP with strong product completeness and clear business relevance. Across code and PPT, the strongest throughline is practical decision support: dashboards, supplier drill-downs, alerts, simulations, reports, logistics views, and admin governance are all aligned to procurement risk management. The backend appears thoughtfully organized around domain routes and services, with auth, RBAC, audit logging, realtime updates, and centralized server-side LLM usage. On the AI side, the team implemented grounded prompts tied to live supplier and alert context, not just generic chatbot behavior, and used LLMs in multiple workflows that map directly to user value: explanation, report drafting, and what-if narrative generation. Security and HITL are also better than average for a hackathon build, especially the admin approval gate for scoring/model changes.

Weaknesses & Gaps

The main recurring gap is between the deck’s framing and the code-backed implementation. The PPT strongly markets a five-agent, continuously integrated, early-detection system, but the code more clearly supports a modular application with LLM-backed features than an explicit multi-agent architecture with autonomous handoffs or orchestration. Persistence is lightweight via lowdb JSON, which is pragmatic for an MVP but limits claims around scale and continuous multi-source monitoring. LLM handling is useful but fairly basic: plain-text outputs, no structured validation, no detected vector-store retrieval, and limited explicit prompt-injection or transparency controls. Privacy/compliance implementation is also under-evidenced relative to the problem’s expectations, and direct proof of external procurement/financial/news integrations, anomaly detection, and measured predictive outcomes is limited.

PPT vs Code Discrepancies

The clearest discrepancy is the “five specialized agents work together” claim on the PPT versus the codebase, which evidences feature-specific routes and services rather than visible agent orchestration. Similarly, the deck implies continuous ingestion of financial reports, delivery logs, and market/news signals, while available code evidence points more to seeded/local datastore workflows than to real external connectors or ingestion pipelines. “Continuous monitoring” is partially supported by WebSocket live updates, but there is no strong evidence of schedulers, background jobs, or autonomous monitoring loops. The presentation also suggests stronger anomaly-detection or predictive analytics than the retrieved code substantiates; the implementation supports composite scoring, trends, and simulation more convincingly than distinct ML/anomaly models.

Innovation Notes

The innovation here is meaningful but pragmatic rather than deeply novel at the systems level. The strongest originality comes from combining grounded LLM assistance with procurement workflows users can act on: supplier explanation, risk-to-dollar what-if simulation, admin review, and logistics/risk visibility. That creates real operational value. However, the more ambitious innovation claims—multi-agent intelligence, advanced anomaly detection, and broad multi-source autonomous monitoring—are not strongly proven in code, so innovation credit is warranted but should remain moderate rather than maximal.

Rule-level Evidence

- ****Problem Statement (PS)**:** `sentinelchain/README.md` explicitly ties screens to “catching supplier disruptions early,” “understanding why,” and “acting on it”; `backend/src/server.js` mounts suppliers, alerts, what-if, reports, chat, scoring, and logistics APIs. Gaps remain for external procurement/financial API integration, sentiment analysis, and outcome metrics.
- ****Backend (BACK)**:** `sentinelchain/backend/src/server.js` mounts modular routes including `auth.routes.js`, `suppliers.routes.js`, `alerts.routes.js`, `whatif.routes.js`, `reports.routes.js`, `chat.routes.js`, `scoring.routes.js`, and `logistics.routes.js`; `backend/src/db/database.js` uses `lowdb` + `JSONFile` with structured default entities and mutation helpers.
- ****Frontend (FRONT)**:** `sentinelchain/frontend/src/App.jsx` includes routes for dashboard, suppliers, supplier detail, alerts, what-if, insights, reports, logistics, and admin; `frontend/src/context/AuthContext.jsx` shows persisted auth flow; `frontend/src/components/ExplainChat.jsx` manages busy state and posts supplier chat requests.
- ****LLM Usage (LLM)**:** `backend/src/routes/chat.routes.js` builds supplier-specific grounded context including supplier identity, risk score, and alerts; prompt instructions constrain scope, style, and allowed topics; `README.md` cites per-supplier chat, what-if narrative, report drafting, and general assistant usage.
- ****AI Engineering (AI_ENG)**:** `README.md` states all Claude calls happen server-side and are grounded in datastore context; `backend/src/server.js` exposes distinct AI-enabled workflows via `/api/whatif`, `/api/reports`, `/api/chat`, `/api/scoring`, `/api/logistics`; fallback behavior was noted as strong when keys/models fail. Static evidence did ****not**** detect vector-store RAG or agentic tool use.
- ****Security & HITL (SECURITY_HITL)**:** `backend/src/routes/scoring.routes.js` contains `/model-review/decision` protected by `requireAuth` and `requireRole("admin")`, explicitly described as a human review gate; `frontend/src/pages/admin/AdminQuality.jsx` reinforces that model updates require sign-off; `backend/src/server.js` uses env-based config and CORS; static checks found no hardcoded secrets, but no validation library was detected, and `src/Utils/jwt.js` includes a dev-secret fallback.
- ****MVP & Outcome (MVP)**:** `backend/src/server.js` starts HTTP/WebSocket services and mounts broad product APIs; `README.md` says the app runs locally with seeded data; `frontend/src/components/Chatbot.jsx` and listed UI screens support a genuinely demoable end-to-end product.
- ****Presentation & Innovation (PRESENTATION)**:** PPT Slide 2 frames the core problem (“Manual supplier monitoring is fragmented and reactive”); PPT Slide 3 presents the five-agent concept; implemented UI labels in `frontend/src/components/AppLayout.jsx` and the README largely support the product story, though not all agent/system claims are equally code-backed.
- ****Innovation & Business Value (INNOVATION)**:** `frontend/src/pages/WhatIf.jsx` demonstrates disruption-duration simulation; `frontend/src/pages/Suppliers.jsx` and `SupplierDetail.jsx` support prioritization and action; the README emphasizes translating risk into dollar impact and decision-ready workflows.

Recommended Judge Questions

1. Your slides describe five specialized agents—where in the code can we see agent orchestration, handoffs, or independent agent behavior versus modular routes/services?
2. What real external data sources are currently integrated, and which supplier/financial/news inputs are still mocked, seeded, or manually maintained?
3. How do you defend against prompt injection or misleading supplier context, especially since LLM outputs appear to be plain text without schema validation?
4. What evidence do you have that the scoring and alerting are predictive or materially useful—any accuracy, false-positive, adoption, or disruption-reduction metrics?
5. How would this architecture evolve from `lowdb` + seeded data into a production-grade continuous monitoring platform?
6. Can you walk through the human-in-the-loop approval flow and explain exactly what changes are blocked until admin sign-off?

Evaluation Confidence & Coverage

- Code ingested: 66 file(s) across 4 layer(s) (FE/BE/DB/OTHER).
- Backend framework recognition: NONE recognized (22 backend-layer file(s) scanned). Static route/integration analysis found no known-framework signal - treat Integration Map findings as unavailable, not negative.
- 3/11 deterministic checks found concrete evidence (PASS/FAIL); 8 had no signal (NOT_DETECTED = absence of evidence, not evidence of absence).
- RAG retrieval: BM25 + embeddings + cross-encoder reranker.
- Knowledge graph: 70 nodes, 66 edges - Graph-RAG context available.